

The Complete Guide to Messaging

Queues, logs and event-driven design: partitions and consumer groups, delivery semantics, the outbox pattern, idempotent consumers, dead-letter queues, backpressure and stream processing.

Queue or log — decide it

Work to be executed once, or events other teams will read later. One question settles it.

Kafka where it counts

Partitions, offsets, consumer groups, ISR and retention — the parts interviews probe.

Exactly-once, honestly

What at-least-once really means, the outbox, idempotency keys, and where duplicates come from.

When it goes wrong

Consumer lag, poison messages, DLQs, rebalance storms, backpressure and replay.

What's inside

Read it front to back once. Part 3 (the outbox and idempotency) is the part that most often separates a senior answer from a plausible one, and Part 6 is what on-call feels like.

1 Queues, logs and why async at all

What asynchrony actually buys, queue versus log, the four patterns, and the coupling you trade away.

2 Inside a log: Kafka mechanics

Topics, partitions and keys, offsets and consumer groups, replication and ISR, retention, compaction, rebalancing.

3 Getting events out of your database

Dual writes and why they break, the outbox pattern, CDC, ordering, and schema evolution with a registry.

4 Delivery semantics and idempotency

At most once, at least once, effectively once; idempotency keys, dedupe windows, and Kafka transactions.

5 Designing event flows

Event notification vs event-carried state vs event sourcing, choreography vs orchestration, sagas, fan-out.

6 Operating it

Consumer lag as the SLI, poison messages and DLQs, retries with backoff, backpressure, rebalance storms, replay.

7 The interview itself

How to introduce a queue without hand-waving, 14 asked-in-real-interviews Q&A, and the red flags to avoid.

★ One-page cheat sheet

The night-before page: the decision table, the numbers, messaging in eight facts.

Part of the HLD concept series. Caching, databases, sharding and replication are separate volumes; Redis Streams and Pub/Sub live in the Redis volume.

How to use this guide

Understand

Each mechanism gets the plain-English version before the vocabulary, and the failure it exists to survive.

Say out loud

The grey “interview line” boxes are the sentences that show you have run one of these systems.

Defend

Async buys decoupling and costs you ordering, debuggability and exactly-once. Name the cost.

THE WORDS, PRECISELY

A **queue** distributes work: one consumer wins each message and it is then gone. A **log** (Kafka, Kinesis, Pulsar) is an ordered, retained sequence many independent consumers read at their own offsets. **Pub/Sub** is a delivery style — fan-out to whoever is listening — and it can be built on either. Using “queue” for all three is the most common vocabulary slip in these interviews.

THE ONE THING TO INTERNALISE

Going asynchronous buys **availability and smoothing**: the producer stops depending on the consumer being up or fast. It charges you in **ordering, duplicates, debuggability and end-to-end latency you can no longer see in one trace**. Every message you add is a distributed system you now own.

Queues, logs and why async at all

PART 1

“I’d put a queue there” is the most casually thrown phrase in system design interviews. Earning the point means saying what it buys and what it breaks.

1.1 What asynchrony actually buys

Decoupling in time

The producer completes even if the consumer is down, deploying or slow. This is the availability argument and it is the main one.

Smoothing bursts

A 10× traffic spike becomes a longer queue instead of a wall of 503s — as long as the work is genuinely deferrable.

Fan-out

One event, many consumers, none of which the producer needs to know about. This is the argument for a log rather than a queue.

Slow work off the request path

Encoding video, sending email, building a thumbnail. The user gets 202 Accepted and a way to check status.

1.2 Queue or log?

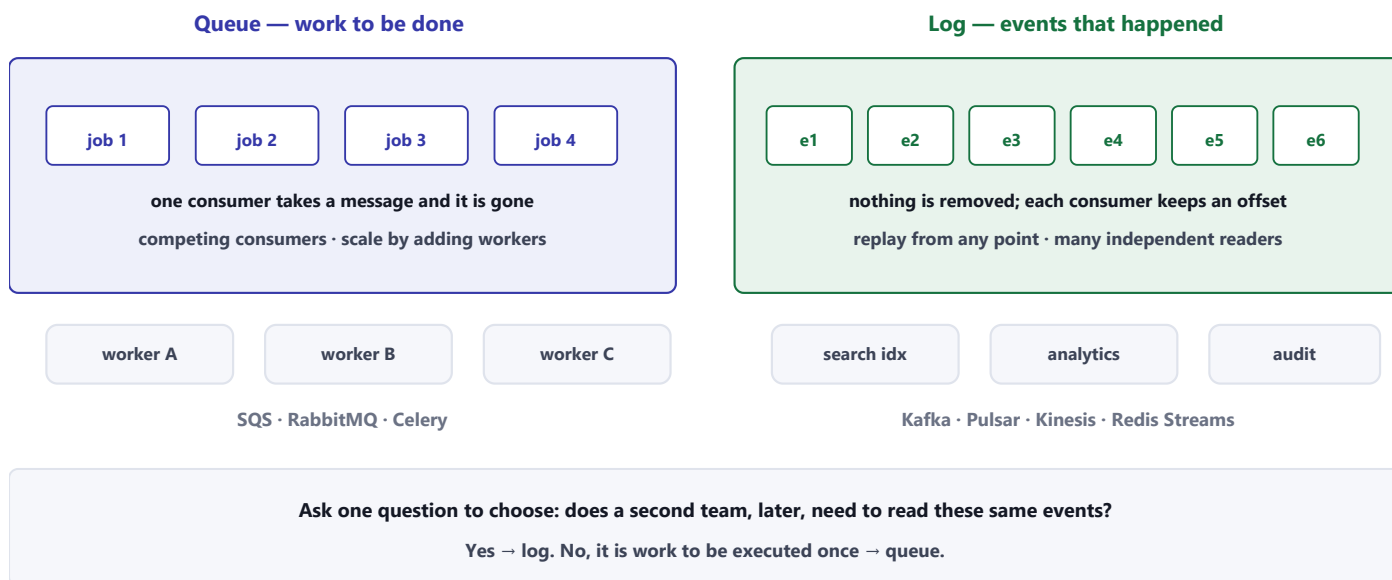


Figure 1.1 — A queue deletes what has been handled; a log remembers and lets each consumer track its own position.

1.3 The four things people actually build

Pattern	Shape	Use it for
Work queue	Producer → queue → competing workers, message deleted on ack	Thumbnails, emails, exports, retries — anything that must happen once
Publish/subscribe	One event delivered to every interested subscriber	Notifying other services that something happened, without knowing who they are
Event log / stream	Retained, ordered, replayable; each consumer holds an offset	Feeding search indexes, analytics, audit, ML features, and rebuilding state
Request/reply over a broker	Correlation id and a reply queue	Rare and usually a mistake — if you need an answer now, call the service

1.4 The costs, said out loud

- **Ordering is only per partition or per key** — a topic has no global order, and assuming one is the classic bug.
- **Duplicates are normal**, because at-least-once delivery is what durable brokers give you. Consumers must be idempotent.
- **Latency becomes invisible** unless you trace it: “the write returned in 20 ms” says nothing about when the effect happened.
- **Failure moves**: instead of a 500 the user sees nothing happen, which is harder to notice and harder to explain.
- **You now operate a broker**: partitions, retention, lag, rebalances and a dead-letter policy are yours.

INTERVIEW LINE

“I would put this behind a queue because the user does not need the result to commit, and because the downstream system being slow should not fail the request. The costs I am accepting are eventual consistency of the effect, duplicate deliveries — so the consumer is idempotent — and the fact that I now need lag monitoring and a dead-letter path.”

Inside a log: Kafka mechanics

PART 2

You do not need Kafka internals to design a system, and you do need enough of them to answer “how does ordering work?” and “what happens when a consumer dies?”.

2.1 Topics, partitions, keys

A topic is split into **partitions**, each an append-only file with monotonically increasing **offsets**. The producer picks a partition — by hash of the message key, or round-robin when there is no key. Two consequences carry most interview answers: **order is guaranteed inside a partition only**, and **the partition count is your maximum consumer parallelism**.

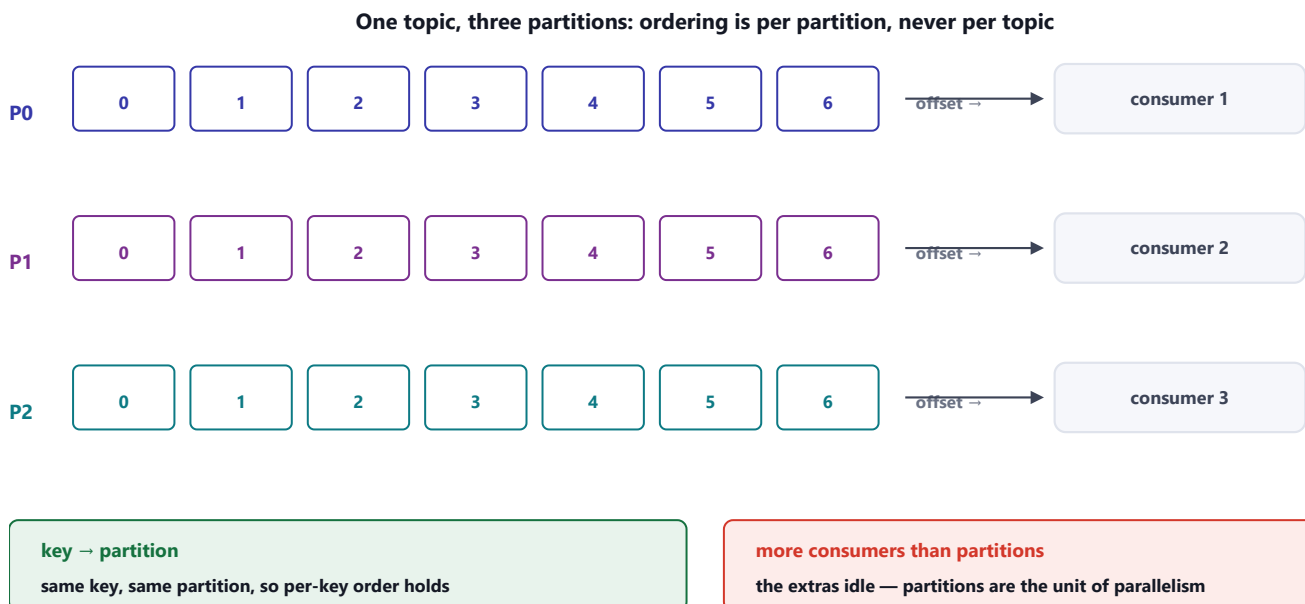


Figure 2.1 — Partitions are the unit of ordering, of parallelism and of assignment. Everything else about a consumer group follows from that.

2.2 Consumer groups and offsets

A group is a scaling unit

Each partition is assigned to exactly one consumer in the group, so adding consumers past the partition count does nothing.

Offsets are committed, not deleted

The consumer records “I have processed up to 4,201”. Restarting resumes there — and committing before processing is how you lose messages.

Several groups, same data

Search indexing, analytics and audit each read the whole topic independently. This is the property a queue cannot give you.

Rebalancing

When a member joins or dies, partitions are reassigned — and processing pauses briefly. Frequent rebalances are usually a too-short session timeout or a slow poll loop.

2.3 Durability: replication and ISR

Each partition has a leader and followers. The **in-sync replica** set is those followers that are caught up; a producer with **acks=all** waits for all of them, so the write survives any single broker loss. **min.insync.replicas=2** with replication factor 3 is the standard production setting: it tolerates one broker down and refuses writes rather than accepting one it cannot keep.

Setting	Meaning	When you would use it
<code>acks=0</code>	Fire and forget	Never, for anything that matters
<code>acks=1</code>	Leader has it	High-throughput telemetry where a broker failure may lose the tail
<code>acks=all + min.insync=2</code>	Leader plus a caught-up follower	The default for business events
<code>unclean.leader.election=true</code>	A lagging replica may become leader	Availability over durability — say it out loud if you enable it

2.4 Retention and compaction

Time or size retention (“keep seven days”) is what makes replay possible — the single most useful operational property of a log. **Log compaction** is the other mode: keep the latest value per key forever, so the topic becomes a rebuildable snapshot of current state. That is how a service bootstraps a cache or a materialised view from the beginning of time without a database dump.

Getting events out of your database

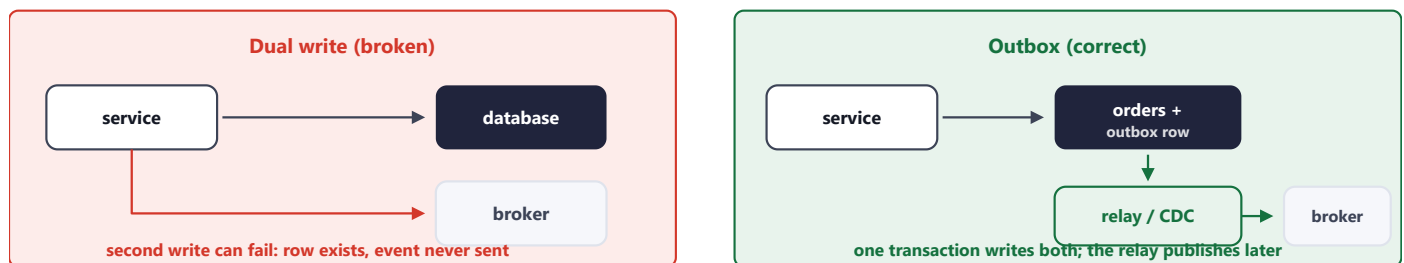
PART 3 · THE SENIOR SIGNAL

“Save the row and publish the event” looks like one line of code and is actually a distributed transaction. How you answer this is the clearest read on whether you have operated an event-driven system.

3.1 Why the dual write is broken

Two independent systems, two writes, no atomicity: the database commit can succeed and the publish fail (a row nobody hears about), or the publish can succeed and the transaction roll back (an event for something that never happened). Retries do not fix it, because the process can die between them.

“Save the order and publish OrderCreated” — atomically



What this buys, and what it still costs

Buy: the row and the event cannot disagree — they commit or fail together.

Costs: the event is published after the commit, so consumers are eventually consistent (milliseconds).

Still at-least-once: the relay can publish twice, so every consumer must be idempotent.

Figure 3.1 — The outbox: one transaction writes both the state and the intent to publish, and a relay ships it afterwards.

3.2 The two correct mechanisms

Transactional outbox Write the domain row and an outbox row in the same transaction. A relay polls the outbox (or tails the log) and publishes, marking rows sent. Explicit, testable, and it works on any database.	Change data capture Read the database's own replication log — Debezium on MySQL binlog or Postgres logical decoding — and turn committed changes into events. Nothing to write in the application, and the log is the source of truth by construction.
---	--

Concern	Outbox	CDC
What you publish	Exactly the event you designed	Row changes; you map them to events downstream
Coupling	Application owns the contract	Consumers see your schema — a real coupling risk
Ordering	Per aggregate, if you key it that way	Per table and transaction, in commit order
Operational load	A relay to run and monitor	A connector, slot/binlog retention, and schema-change handling
Failure mode	Outbox grows if the relay stalls	Replication slot grows and can threaten the database

3.3 Schema evolution

Events outlive the code that wrote them, so the contract needs rules: add fields as optional, never repurpose a field, version the schema, and register it (Avro/Protobuf/JSON Schema in a registry) so producers cannot ship a breaking change. The one-liner: **“consumers must tolerate unknown fields and missing optional ones — that is what makes independent deploys possible.”**

Delivery semantics and idempotency

PART 4

The three-word answers — at most once, at least once, exactly once — are easy. The points come from knowing which one you actually get, and what you must build because of it.

4.1 The three semantics

Where the duplicate and the loss actually come from

At most once	At least once	Effectively once
ack before processing crash → message lost metrics, fire-and-forget logs	ack after processing crash → redelivered the default everywhere	at-least-once + dedupe idempotency key or txn payments, orders, ledgers

“Exactly once” in the wild
Kafka transactions give exactly-once between Kafka topics. The moment an external system is involved — a database, an email, a payment API — you are back to at-least-once plus an idempotent consumer.

Figure 4.1 — Ack before processing loses; ack after processing duplicates. There is no third option without dedupe.

4.2 Making a consumer idempotent

Natural idempotence

“Set status to SHIPPED” is safe to apply twice. Prefer designing operations this way over adding machinery.

Dedupe table

Store the event id (or a hash) with a unique index and insert it in the same transaction as the effect. A duplicate hits the constraint and is dropped.

Idempotency key at the API edge

The client sends a key; you store the first response against it and replay that response for retries. This is the Stripe pattern.

Conditional writes

Version numbers, sequence numbers, or “update where status = PENDING” — the effect applies once because the condition only matches once.

4.3 “Exactly once” — what is really on offer

Kafka’s transactions give exactly-once **within Kafka**: a consume-transform-produce loop can commit offsets and output atomically. The moment the effect leaves the broker — a database row, an email, a card charge — you are back to at-least-once delivery with an idempotent consumer. The honest phrase is **effectively once**, and using it is a senior tell.

INTERVIEW LINE

“I would design for at-least-once, because that is what a durable broker gives me, and make the consumer idempotent: the event id goes into a dedupe table in the same transaction as the effect, so a redelivery is a constraint violation and not a double charge. If both ends were Kafka topics I could use transactions for true exactly-once, but the moment I write to an external system that guarantee ends.”

4.4 Ordering, when you genuinely need it

- **Key by the entity** — order id, account id, conversation id — so all events for that entity land in one partition and are processed in order.
- **Sequence numbers** in the payload, so a consumer can detect a gap or an out-of-order arrival rather than trusting the transport.
- **Single-threaded per key** inside the consumer: parallelise across keys, never within one.
- **Accept that retries reorder.** A failed message sent to a retry topic will arrive after later messages for the same key — which is exactly why the handler should be commutative or version-checked.

4.5 The idempotency table, concretely

consumer, one transaction

```
BEGIN;  
  INSERT INTO processed_events(event_id)  
    VALUES ($1);          -- unique index  
  UPDATE orders SET status = 'PAID'  
    WHERE id = $2 AND status = 'PENDING';  
COMMIT;
```

A duplicate delivery fails the insert, the transaction aborts, and nothing happens twice. The condition on status makes it safe even if the row was already advanced.

what to keep, and for how long

```
event_id      uuid primary key  
processed_at  timestampz  
-- retention: longer than the broker's  
-- redelivery window, then partition  
-- by day and drop old partitions
```

Bound the table or it becomes the next scaling problem. A dedupe window wider than the broker’s retention is wasted; narrower is a hole.

Designing event flows

PART 5

Once you have a broker, the design question becomes what goes in the message and who is in charge — and those two choices determine how coupled your services really are.

5.1 What goes in the event

Style	Payload	Trade-off
Event notification	"Order 1183 changed" plus an id	Tiny and loosely coupled; every consumer must call back for details, so you have moved the load rather than removed it
Event-carried state	The fields consumers need, denormalised	No callback, so consumers work while the producer is down; the schema is now a contract and payloads get big
Event sourcing	Every state change, as the system of record	Perfect audit and time travel; you now own snapshots, replay performance and schema versioning forever

5.2 Who is in charge: choreography or orchestration

Choreography Each service reacts to events and emits its own. No central brain, easy to extend — and no single place to see "where is order 1183?", which makes debugging a log-joining exercise.	Orchestration A saga orchestrator owns the state machine and calls each step. One place to see progress, retry and compensate; that component is now critical and must itself be durable.
---	---

The rule of thumb worth saying: **choreography for notifications and fan-out, orchestration for business transactions with compensation** — anything where a partial failure needs a defined reversal.

5.3 Sagas in three sentences

A saga is a sequence of local transactions where each step has a compensating action, used because a transaction cannot span services. Every step needs an idempotency key, the state machine must be persisted so it can resume after a crash, and compensation is business logic — "refund the payment", not "roll back the transaction". Interviewers listen for that third sentence.

Operating it

PART 6

Everything in this part is what being on call for an event-driven system feels like. Volunteering two of these is worth more than naming three brokers.

6.1 Consumer lag is the SLI

Lag — the difference between the newest offset and the committed one — is the number that tells you whether the system is keeping up, and it is the one to alert on. Watch the *derivative* too: flat lag at 50,000 is a system in balance, while steadily growing lag is a system that will not recover without more consumers or fewer events. Alert on lag in **time** (“more than two minutes behind”), because that is what the product cares about.

6.2 Poison messages and dead letters



The failure to name: **a poison message with in-place infinite retry blocks its whole partition**, so one malformed event stops every key that hashes to it. That is the outage a DLQ prevents, and “what happens to a message you can never process?” is a standard follow-up.

6.3 Backpressure

The broker absorbs, until it does not

Retention is finite. A consumer down longer than retention loses data — a real incident, not a theoretical one.

Bound everything

Prefetch limits, in-flight caps, and a bounded internal work queue. Unbounded buffering converts a slow consumer into an OOM.

Shed or degrade

Drop low-value events (telemetry sampling) rather than falling behind on the ones that matter. Separate topics make that possible.

Scale the unit that matters

More consumers only help up to the partition count — so partition count is a capacity decision made in advance.

6.4 Replay, the superpower and the footgun

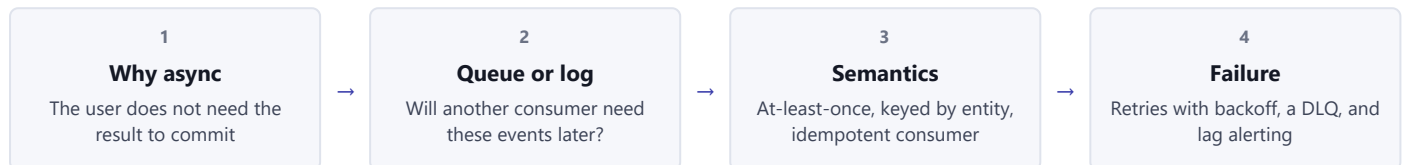
Replaying a retained log rebuilds a search index, backfills a new service, or recovers from a bad deploy — and it re-emits side effects unless consumers are idempotent and external calls are guarded. Design for it explicitly: a replay flag, a dedupe window wide enough to cover the replay, and a way to throttle so the replay does not take the database down.

The interview itself

PART 7

The queue is usually the easiest box to draw and the easiest place to lose points. These four beats keep it credible.

7.1 The four-beat answer



7.2 The five sentences that earn points

- "Ordering is per partition, so I key by order id and process one key at a time."
- "I would not dual-write; the event goes in an outbox row in the same transaction, and a relay publishes it."
- "At-least-once is what I get, so the consumer dedupes on event id inside the same transaction as the effect."
- "A poison message with infinite retry blocks the partition, so failures go to a retry topic and then a DLQ."
- "Consumer lag in seconds is the SLI, and partition count is my parallelism ceiling."

INTERVIEW LINE

"I would use a log rather than a queue here, because the notification service is not the only future consumer — search indexing, analytics and audit will all want the same events, and a log lets them read independently and replay. The costs are retention capacity and the fact that consumers must be idempotent, which they have to be anyway."

7.3 Fourteen questions with model answers

Q1 Queue or Kafka — how do you choose?

One question: will anyone other than this consumer, now or later, need these events? If yes, a log — retained, ordered per partition, with independent offsets — so search, analytics and audit can each read the whole stream and replay it. If it is work to be executed exactly once by whichever worker is free, a queue is simpler and the operational surface is smaller.

Q2 How does Kafka guarantee ordering?

Only inside a partition. A partition is an append-only sequence with monotonic offsets, and one consumer per group owns it — so events with the same key, hashed to the same partition, are processed in order. A topic has no global order, so if I need per-entity ordering I key by that entity, and if I need global ordering I have a single-partition topic and no parallelism.

Q3 Save an order and publish an event. How do you do it atomically?

Not with a dual write — the commit can succeed and the publish fail, or the reverse. I write the order and an outbox row in the same transaction, and a relay reads the outbox and publishes, marking rows sent; or I skip the outbox and use CDC on the database log. Either way the event cannot exist without the row. It is still at-least-once, so consumers dedupe.

Q4 Is exactly-once delivery possible?

Not end-to-end, in general. Kafka transactions give exactly-once between Kafka topics by committing offsets and output atomically. As soon as the effect is external — a row, an email, a charge — you get at-least-once delivery plus idempotent processing, which is what people mean when they say effectively once. The mechanism is a dedupe key stored transactionally with the effect.

Q5 A consumer keeps crashing on one message. What happens and what do you do?

With in-place retry it blocks the partition: every message behind it stops, including other keys that hash there. So: a few retries with exponential backoff and jitter, then move the message to a retry topic so the main partition flows, then to a dead-letter queue with the error and enough context to reproduce. Alert on DLQ depth, and make replay from the DLQ a normal operation rather than a script someone writes during an incident.

Q6 How do you scale consumers?

Add consumers up to the partition count and no further — partitions are the unit of assignment, so extra consumers idle. That means partition count is a capacity decision you make in advance, with headroom, because increasing it later changes key-to-partition mapping and therefore breaks per-key ordering for in-flight data. Inside a consumer, parallelise across keys, never within a key.

Q7 What is consumer lag and what do you alert on?

The gap between the newest offset and the one the group has committed. I alert on lag expressed in time — “more than two minutes behind” — because that is what the product notices, and I watch whether it is flat or growing: flat lag is balance, growing lag is a system that will not recover on its own. I would also alert on a group that stops committing at all, which flat lag can hide.

Q8 The consumer was down for six hours. What now?

First check retention: if the topic keeps seven days I have simply got a backlog, and the questions are how fast I can drain it and whether processing it late is harmful. I would scale consumers to the partition ceiling, consider a temporary separate group for the backlog, and throttle so the downstream database survives. If the outage exceeded retention, data is genuinely gone and I need a backfill from the source of truth — which is the argument for retention longer than your worst realistic outage.

Q9 Event notification or event-carried state?

Notification — an id and a change type — keeps payloads small and coupling low, but every consumer calls back for details, so I have moved load rather than removed it, and consumers cannot work while the producer is down. Carried state makes consumers independent at the price of a schema contract and bigger messages. I would carry state for cross-team consumption and notify inside a team that owns both sides.

Q10 Design the notification fan-out for a social app.

Producers publish one **PostCreated** event to a log, keyed by author. A fan-out consumer group expands it to followers and writes per-channel jobs to separate topics — push, email, in-app — because their rate limits and failure modes differ. Each channel worker is idempotent on (user, post, channel), retries with backoff, and dead-letters after a bounded number of attempts. Celebrity authors get the hybrid treatment: no fan-out on write, merged on read.

Q11 Do you need a broker, or would a database table do?

A table with **SELECT ... FOR UPDATE SKIP LOCKED** is a genuinely good queue at low rates: transactional with your data, no new infrastructure, easy to inspect. It stops being good at high throughput, with many consumers, when you need fan-out or replay, or when the dead tuples start hurting. Saying this out loud is a strong signal — reaching for Kafka to send a thousand emails a day is over-engineering.

Q12 How do you evolve an event schema without breaking consumers?

Additive changes only: new fields optional, never repurpose or remove a field in place, and version the event type when the meaning changes. Enforce it with a schema registry and compatibility checks in CI, so a producer physically cannot ship a breaking change. Consumers ignore unknown fields, which is what lets both sides deploy independently.

Q13 When does a retry make things worse?

When it is immediate, unbounded, or synchronised. Immediate retries hit a struggling dependency hardest; unbounded retries turn a transient error into an infinite loop; and retries across a whole fleet at the same interval produce a thundering herd. So: exponential backoff with jitter, a retry budget or circuit breaker, and idempotency so the retry is safe at all. Also worth naming: retries reorder events for a key.

Q14 What would make you not use async here?

When the user needs the answer to proceed — a payment authorisation, a login, a seat reservation. Making that asynchronous does not remove the wait, it just hides it and adds a status-polling problem. Also when the operation is genuinely simple and synchronous is easier to reason about: a queue adds a broker, duplicates, ordering rules and lag monitoring, and it should earn that.

7.4 Red flags interviewers listen for

Saying this	Says this about you
"Kafka guarantees ordering"	Only per partition — and that distinction is the whole design
"Exactly-once, so no dedupe needed"	Has not read the small print at the system boundary
Dual-writing DB and broker in application code	The two will disagree, and nothing will notice
No answer for a message that always fails	One poison message can stall a partition
Unbounded in-place retries	Turns a transient error into a permanent outage
More consumers than partitions	The extras idle; parallelism is capped by partitions
No retention or lag numbers	Has not operated a broker
"We'll use a queue" for a synchronous user need	Hides the wait instead of removing it
Event payload = the whole database row, versioned by nothing	Consumers break on the next migration
Kafka for a thousand jobs a day	Over-engineering; a table with SKIP LOCKED was the answer

7.5 Real systems worth name-dropping

LinkedIn → Kafka

Jay Kreps' *The Log* is still the clearest argument for a replayable log as the backbone of data integration. Cite it for "why a log, not a queue".

Debezium & the outbox

The industry-standard CDC connector, and the pattern write-up that made "outbox" common vocabulary. Cite it for atomic state-plus-event.

Stripe idempotency keys

A public API contract built entirely on "retry safely". The canonical reference for idempotency at the edge rather than deep in a consumer.

Uber & DoorDash event platforms

Public write-ups on Kafka at scale: tiered storage, consumer-lag SLOs, DLQ tooling. Good for showing you have read operational accounts, not just docs.

One-page cheat sheet

The night-before page. If you remember only this, you can still hold a good conversation about async.

The numbers

one Kafka broker ~ 100 MB/s · partitions = your consumer parallelism · retention beyond your worst outage

Alert on lag in time, not messages. Keep 30–40% partition headroom, because repartitioning later breaks per-key ordering. A dedupe window must be wider than the broker's redelivery window.

The decisions

Question	Default answer
Async at all?	Only if the user does not need the result to commit
Queue or log?	Log if anyone else will ever want these events
Partition key?	The entity whose order matters: order id, account, conversation
Delivery?	At-least-once, with an idempotent consumer
Dedupe?	Event id in a unique index, inserted in the effect's transaction
DB + event?	Outbox in one transaction, or CDC. Never a dual write
Retries?	Bounded, exponential, jittered → retry topic → DLQ
Payload?	Carry state across teams; notify inside one
Saga?	Orchestration for business transactions, choreography for fan-out
Alerting?	Consumer lag in seconds, DLQ depth, and commit staleness

Messaging in eight facts

Async buys availability

The producer stops depending on the consumer being up. That is the whole argument.

Ordering is per partition

Key by the entity that needs order, and process one key at a time.

Partitions cap parallelism

More consumers than partitions do nothing. Decide the count with headroom.

At-least-once is what you get

So idempotency is not optional. Dedupe transactionally with the effect.

Dual writes always diverge

Outbox or CDC. One transaction must cover the state and the intent.

A poison message blocks a partition

Bounded retries, then a retry topic, then a DLQ someone actually watches.

Lag is the SLI

In seconds. Flat lag is balance; growing lag never recovers by itself.

Retention is a data-loss boundary

A consumer down longer than retention has lost data. Size it for your worst outage.

The failure modes

Name	Mechanism	Primary fix
Poison message	A message that always fails, retried in place	Bounded retry → retry topic → DLQ
Duplicate effects	Redelivery after a partial success	Idempotency key in the same transaction as the effect
Lost event	Dual write, or offset committed before processing	Outbox/CDC; commit offsets only after the effect
Rebalance storm	Slow poll loop or short session timeout	Tune timeouts, shorten processing, use cooperative rebalancing
Backlog past retention	Consumer down longer than the retention window	Longer retention, lag alerts, and a backfill path from the source
Reordering by retry	A retried message arrives after later ones	Version checks or commutative handlers; never assume transport order

IF YOU SAY NOTHING ELSE, SAY THIS

"I would go asynchronous where the user does not need the result to commit, and I would use a log rather than a queue whenever a second consumer will ever want the same events. Ordering is per partition, so I key by the entity that needs it. Delivery is at-least-once, so the consumer dedupes on event id in the same transaction as the effect, and the event itself comes out of an outbox rather than a dual write. Failures retry with backoff and end in a dead-letter queue, and consumer lag in seconds is the number I would put on a dashboard and alert on."

Sources & further reading — Jay Kreps, *The Log: What every software engineer should know about real-time data's unifying abstraction*; the Apache Kafka documentation on partitions, consumer groups, ISR, transactions and log compaction; Debezium documentation and the transactional-outbox pattern write-up; Chris Richardson's microservices patterns (saga, outbox, CQRS); Martin Kleppmann, *Designing Data-Intensive Applications*, chapters 11 and 12; Stripe's API documentation on idempotency keys; Confluent's articles on exactly-once semantics; AWS documentation on SQS visibility timeouts, FIFO queues and DLQs; Uber and DoorDash engineering blogs on operating Kafka at scale.